

Workload-Aware Deployment Pattern Selection for Cloud-Native Commerce Systems

Koteswararao Nallabothu
Lead Software Engineer
Lowe’s
Charlotte, NC, USA
Nallabothukoteswar@gmail.com

Naresh Alapati
Principal Cloud Architect
Walmart
Bentonville, AR, USA
narencachy@gmail.com

Abstract—Commerce platforms combine high-concurrency request serving with effects that must not be repeated or applied out of order. We present a workload-aware method that separates three decisions: serving topology, effect discipline, and release mechanism. Its Highlander policy, named for the “there can be only one” constraint, establishes a single effective writer per conflicting resource at participating sinks, rather than exclusive physical execution. The method distinguishes operation-key deduplication from epoch fencing: uniqueness suppresses repeated effects; a sink version check blocks regressions when source order is available, while fencing protects authority-ordered publications. An atomic ownership transaction binds a lease holder to its epoch, preventing a delayed former holder from obtaining a newer token while a successor’s lease is valid. We give a decision algorithm, a sink safety argument, and takeover bounds with explicit timing assumptions. A Kubernetes-hosted evaluation protocol isolates eight mechanism combinations across duplicate-effect and ordered-publication workloads, records wasted work, and accounts for uncertainty in fault timing. An illustrative capacity calculation requires seven replicas for an assumed 600 requests/s peak, 120 requests/s per-replica capacity, and a 20% reserve. A scripted SQLite sink-rule study confirms that a version check rejects regressions under its declared order and that a co-located fence rejects former-epoch effects. A remote, per-feed fence still admits old-epoch effects after a successor grant until the successor’s first write reaches that sink. These scheduled mechanism outcomes are not production performance or failure-rate estimates.

Index Terms—fencing, idempotency, leader election, Kubernetes, deployment, reliability

I. INTRODUCTION

An inventory worker can resume after a pause and publish feed version 7 after another worker has published version 8. Unique operation keys do not prevent this rollback: the publications have different identities. A sink conditional on a usable source version does prevent it without leadership. Conversely, a unique key can suppress duplicate loyalty accrual without a leader. These examples expose two correctness requirements that must be evaluated separately. Serving capacity is independent of both. A checkout API can require many active replicas and a provider idempotency key for payment capture at the same time.

Rolling, blue-green and canary methods control how versions receive traffic [1], [2], [3]; they do not establish exclusive effect ownership. We use *Highlander* as an operational policy

name, not a Kubernetes feature or a new consensus algorithm. It denotes one effective writer per conflict key at a sink that checks epochs when order follows ownership or the sink cannot compare source versions. Old and new processes may execute simultaneously, but a stale epoch must not change protected state.

Our contributions are (i) a three-lane decision procedure combining capacity, availability, effect semantics and release risk; (ii) an ownership protocol whose acquisition atomically binds holder and epoch, together with a sink safety argument; and (iii) an ablation protocol that distinguishes duplicates, version regressions, epoch regressions, late accepts and rejected work against a conditional-version baseline. Sections II–IV give context, selection and protocol. Section V evaluates the protocol and deployment trade-offs. Sections VI–VII discuss applicability and future work.

II. BACKGROUND AND RELATED WORK

Leases bound ownership in time [4]; Chubby [5], ZooKeeper [6] and Raft [7] provide coordination and replicated-state foundations. They do not make a third-party payment or notification service transactional. Fencing tokens [8], retry-safe effects [9] and compensating transactions [10] address different boundaries. A token orders writers; an operation key identifies a repeated request; a saga compensates committed work. None is interchangeable with the others.

Borg, Omega and later Borg traces demonstrate heterogeneous cluster scheduling [11], [12], [13]. MillWheel and Photon offer stronger processing semantics within their own state and output models [14], [15]; those guarantees do not automatically cover arbitrary external APIs. Continuous-experimentation research [16] and canary analysis [17] motivate staged exposure and version-specific metrics. Kubernetes supplies rolling updates [1], Jobs [18], readiness [19], Leases [20] and autoscaling [21]; Argo adds progressive delivery controls [22]. We do not claim to invent leases, fencing or canary releases. Table I positions the systematization.

III. SYSTEM MODEL AND PATTERN SELECTION

Let λ_{peak} be peak requests/s, μ the measured safe throughput of one replica at a declared request mix and latency

TABLE I
SCOPE RELATIVE TO PRIOR WORK

Work	Primary scope	Selection contribution
Leases / locks	Ownership	No topology/release rule
Keys / sagas	Retry/compensation	No capacity rule
Clusters	Placement/resources	No sink policy
Canary	Exposure/metrics	No epoch guarantee
This work	Three decisions	Rule and ablations

Input: servesRequests, $\lambda_{\text{peak}}, \mu, h, R, \text{RTO}, D, \rho, \text{SLO}, E, f, T$.

Output: (serving topology, effect discipline, release).

- 1) If servesRequests and (μ or R unknown), return MEASURE_FIRST for the component.
- 2) **Serving:** If servesRequests, compute (1)–(2). If $n_{\text{req}} > 1$ or (2) fails, select active-active; otherwise single-active is eligible after risk acceptance. If not serving, set serving topology to N/A.
- 3) **Effect:** Independently classify E : none $\rightarrow$ no effect guard; keyable/local transaction $\rightarrow$ unique operation key; source-versioned state $\rightarrow$ conditional version write; authority-ordered state $\rightarrow$ per-key lease and epoch fence; external non-idempotent effect $\rightarrow$ provider key, outbox and reconciliation.
- 4) Combine disciplines if effects overlap. If $D = 0$ and required sink/provider protection is unavailable, block the effect path pending redesign; if $D > 0$, record and monitor an explicit duplicate policy. Ownership may also reduce contention and rejected preparation work.
- 5) **Release:** Independently, high behavioral ρ selects canary analysis gates; high rollback urgency selects blue-green with compatible state; otherwise select rolling with availability controls.

Algorithm 1: Deployment Pattern Selection

objective, and $h \in [0, 1)$ the reserve fraction. Approximately balanced serving requires

$$n_{\text{req}} = \left\lceil \frac{\lambda_{\text{peak}}}{\mu(1-h)} \right\rceil. \quad (1)$$

This planning condition is not a latency guarantee: dependencies, skew and queueing can invalidate the assumed μ . For rare, non-overlapping failures, let f be expected failures per period T and R mean takeover time. A single-active service passes the availability screen only if

$$fR \leq (1 - \text{SLO})T, \quad R \leq \text{RTO}. \quad (2)$$

Tail-sensitive objectives require a percentile or correlated-failure analysis beyond this expectation. A standby may reduce recovery time but contributes no serving throughput.

Let D be tolerated duplicate effects per period, ρ the predeclared release-risk category, and E the effect semantics. Algorithm 1 evaluates every component in three lanes. A composite application receives both a serving plan and a worker/effect plan; effect handling never substitutes for capacity sizing. Unknown serving measurements return MEASURE_FIRST, avoiding arithmetic on a missing μ . A batch-only component has no requests/s sizing branch. Figure 1 summarizes the lanes.

For illustration, $\lambda_{\text{peak}} = 600$, $\mu = 120$ requests/s and $h = 0.20$ give $\lceil 600/96 \rceil = 7$. Six replicas provide 576 requests/s after reserve and fail the condition; seven provide 672 requests/s, or 72 requests/s beyond the assumed peak. These are declared inputs and arithmetic, not observations. The availability screen remains symbolic until f and R are measured. Table II gives composite outcomes.

TABLE II
WORKLOAD-TO-PATTERN MAPPING

Workload	Serving plan	Effect discipline
Checkout API	Sized active-active	Provider key; reconcile capture
Internal UI	(1)–(2) determine eligibility	Keyed local writes where needed
Ordered feed job	N/A	Source version check; lease + fence for owner-defined order or costly contention
Export API + job	Active-active API	Per-export key; bounded workers

IV. HIGHLANDER ARCHITECTURE AND SAFETY

A. Effect boundary and atomic ownership

Figure 2 separates requests from asynchronous work. For order-confirmation email, an outbox durably records intent, while a provider key or reconciliation addresses an ambiguous acknowledgment. Payment capture needs provider idempotency. An inventory feed with usable source versions can use a conditional version write; a search-index alias whose order is determined by its current publisher needs epoch fencing. Loyalty accrual often admits a unique business operation key. A system with several sinks needs protection at each sink; one database transaction cannot confer exactly-once behavior on an uncooperative external API.

The planned testbed binds ownership and epoch in PostgreSQL: an owner row ($k, \text{holder}, e, \text{expires_at}$) is changed by a conditional update on the database clock. Acquisition assigns holder, increasing epoch and expiry in the *same* transaction. Renewal requires matching holder and epoch and an unexpired row. Each process incarnation has a UUID. A sink-fenced write locks that row and atomically checks authority with its ledger/state update.

We call this the *lease-then-epoch race*: W1 acquires a Kubernetes Lease, pauses before requesting a separate epoch, and resumes after W2 has acquired a lease and token. Giving W1 a later token now legitimizes a zombie. Our protocol has no separate token-allocation operation: W1’s delayed conditional acquisition is rejected while W2 owns an unexpired row. A pause after successful acquisition leaves W1 with its old epoch. Figure 4(b) illustrates this fix. A Kubernetes-only alternative must bind a holder change and epoch to the same compare-and-swap update, preserve monotonicity across object recreation, and use unique incarnation identities; it is not the protocol evaluated here.

B. State transitions and safety property

Table III and Fig. 3 describe local control states. “Active” is a client’s belief, not proof that other processes stopped. Shutdown and uncertain renewal enter Draining. Crashes from Candidate, Active or Draining enter Failed; recovery returns to Standby only after durable-state reconciliation.

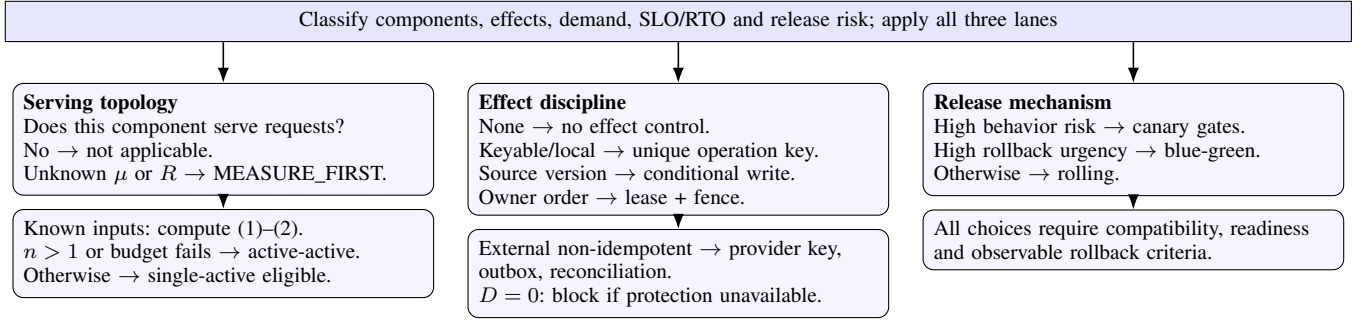


Fig. 1. Three independent selection lanes. A checkout API can receive active-active serving, provider idempotency controls and a canary release simultaneously. The measurement exit applies only to request-serving components.

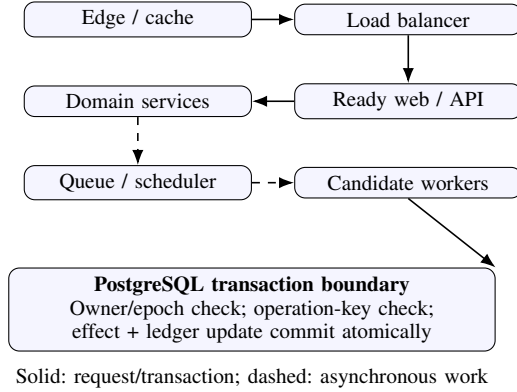


Fig. 2. Reference architecture. The asynchronous branch is dashed. In the testbed, ownership validation, ledger insertion and effect mutation share a PostgreSQL transaction boundary.

TABLE III
WORKER TRANSITIONS

From	Event [guard] / action	To
Standby	work [enabled] / acquire	Candidate
Candidate	grant [atomic CAS wins] / begin	Active
Candidate	denied [valid owner] / back off	Standby
Active	renew fails [uncertain] / stop admission	Draining
Active	shutdown [requested] / checkpoint	Draining
Draining	settled [durable checkpoint] / stop	Standby
Candidate, Active, Draining	crash [any] / no further action	Failed
Failed	recover [reconciled] / rejoin	Standby

For the combined discipline, let $e_{\text{current}}(k)$ be the durable current epoch and $\mathcal{K}(k)$ the committed operation keys. The participating sink enforces

$$\text{accept}(k, e, op) \Rightarrow e \geq e_{\text{current}}(k) \wedge op \notin \mathcal{K}(k). \quad (3)$$

In the co-located fenced conditions, the implementation strengthens the epoch check to equality with the current holder

and valid expiry; tokens are never accepted merely because a client presents a larger number. The effect and ledger update commit atomically. Monotonic epoch issuance and a serialized check-and-write reject a former epoch after its successor is installed. Atomic uniqueness rejects an already committed operation key even within the same epoch. This proves neither cross-sink atomicity nor exactly-once external effects, and does not forbid an earlier valid write before the successor acquires ownership. In fence-only C2f, only the epoch clause is enforced; in unique-only C1u, only the operation-key clause is enforced. C1v instead checks a monotone data version at the sink without acquiring ownership.

C. Time and failure assumptions

A client-side safety deadline is $t_{\text{renew}} + L - \delta$: effects must finish before that deadline. For Kubernetes Lease clients, expiry is judged from local observations, not an API-server expiry clock. Here δ conservatively includes clock-rate drift between candidates, observation delay and write-to-commit time [23]. No finite client-only margin proves safety under unbounded pauses. In our PostgreSQL implementation the database clock decides expiry and the sink transaction decides authority; client stopping is an optimization, not its safety foundation.

Measured from the last successful renewal, acquisition and activation satisfy $R_{\text{renew}} \leq L + P + S$ under bounded coordination and scheduling. From a crash Δ after renewal, time to the first accepted write instead satisfies

$$R_{\text{accept}} \leq \max(0, L - \Delta) + P + S + W. \quad (4)$$

P covers detection/polling and coordinator response; S covers activation and work preparation through the first successful submission; W covers sink acceptance. A drain term G is added only when a required serialized drain extends beyond expiry, not when it already occurred inside L . Unbounded partition or sink delay invalidates a finite bound. Expand/contract schema changes and backward-compatible checkpoints remain necessary: neither a traffic switch nor a ready Pod guarantees a safe worker-state rollback [19].

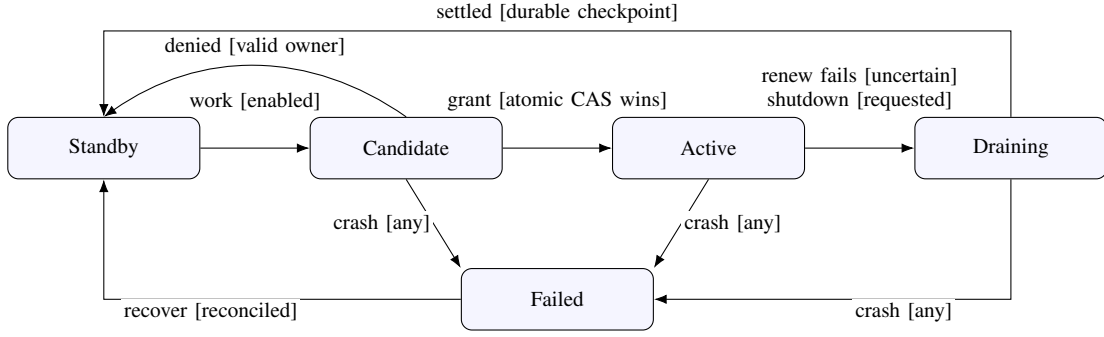
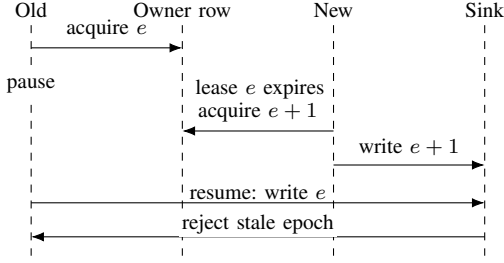
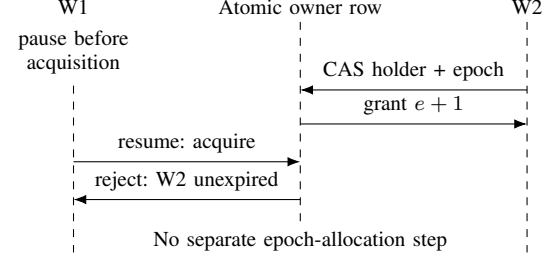


Fig. 3. Local states with event [guard] labels. The curved denial edge returns Candidate to Standby.



(a) Stale effect after lease expiry



(b) Atomic acquisition rejects a delayed contender

Fig. 4. Two distinct pause hazards. (a) A sink rejects an old epoch after a successor’s publication. (b) Binding holder and epoch in one conditional transaction prevents the separate-allocation race.

TABLE IV

MECHANISM ABLATIONS. LEASE IS A POSTGRESQL OWNER-ROW LEASE; CO-LOCATED FENCE CHECKS HOLDER, EPOCH AND EXPIRY; C3R CHECKS ITS SINK EPOCH.

Condition	Lease	Fence	Version	Unique	Replay
C1	—	—	—	—	—
C1u	—	—	—	Yes	—
C1v	—	—	Yes	—	—
C2	Yes	—	—	—	—
C2f	Yes	Yes	—	—	—
C3	Yes	Yes	—	Yes	—
C3r	Yes	Remote	—	Yes	—
C4	Yes	Yes	—	Yes	Yes

V. EVALUATION

A. Testbed and scope

The planned full protocol uses a two-node kind cluster on one host, PostgreSQL, three Python candidates, and a CPU-limited HTTP service. The co-located sink uses one database; the restricted remote sink uses a separate schema and role with no access to ownership; Kubernetes hosts and restarts processes. Run manifests capture images, Kubernetes/node metadata, configuration, fault parameters and seeds. The eight mechanism combinations are listed in Table IV.

B. Methodology and hypotheses

The planned cluster comparison at $b = 10$, $d = 1$ s has 30 planned trials per condition and workload (C1v: ordered only), with 1,000 logical items and randomized condition order. The

separate sweep predeclares C1, C1u, C2, C1v (ordered only), and C3r, with 10 trials per cell for $b \in \{1, 10, 100\}$ and successor-to-resume delay $d \in \{0, 1, 5\}$ s; C3r sweep cells use the after-grant trigger and ordered workload. A C2 ordered-workload smoke trial with $b \geq 10$ checks that a verified stop creates nonzero exposure. The duplicate workload reuses business operation keys after replay. The ordered workload publishes a source-sequenced version and a distinct operation key for each publisher/item pair. In C1, C1u and C1v, the source assigns a monotone generation sequence independently of any lease; a lower generation resuming after a higher publication is a version regression. The same source version construction is used in every condition. A version regression is an accepted write whose source version is lower than the stored version; this metric applies to every condition. An epoch (authority) regression is an accepted write whose epoch is lower than the highest already applied epoch; this metric applies only to lease conditions. Instrumentation records pre-write state, so neither metric infers ordering from arrival timestamps alone.

A configurable in-flight batch is prepared before a worker self-stops with SIGSTOP. The Pod shares a process namespace with its sandbox, so the worker is not PID 1; the runner verifies process state T and a stopped database heartbeat before accepting the fault [24]. After a successor accepts a write (or, for paired C3/C3r, after its committed grant but before its first sink write), SIGCONT releases the captured batch without revalidating its already-admitted effects; the sink is the tested barrier. Batch size is recorded because

it changes exposure. Hypotheses are that C1u and C3 both prevent repeated-key duplicates, whereas C1v prevents version regressions under the specified source-version workload and fenced conditions prevent epoch regressions. Epoch regressions represent authority-ordered effects, where no comparable data version exists and a version check cannot apply; C2 versus C2f/C3 isolates the fence. Fence-only C2f can still accept repeated operation keys. C4 tests replay-response stability, not a lower duplicate count than C3. C1v should have zero version regressions yet more rejected attempts and preparation CPU than C3 because three uncoordinated workers compete. Leadership can reduce wasted work even where source-version checking suffices for safety. The C3r fence tracks e_{current} per ownership key, here the whole feed, not per item: a late accept is an older-epoch effect accepted after a successor grant but before that successor writes to the remote sink. The grant-acknowledgment timestamp conservatively confirms the committed grant, and a delayed first write exposes this interval equally to C3 and C3r. In after-grant trials the successor waits $w_{\text{first}} = 2$ s before its first sink write. C3r late accepts are expected to be positive for $d \in \{0, 1\}$ s and zero for $d = 5$ s; C3 should have zero at every d .

A separate replay series injects lost acknowledgments (5% of accepted calls; the worker retries the same operation key after 50 ms) without a pause fault, comparing C2f, C3 and C4 on the duplicate workload with 30 trials each. Retries are classified as accepted duplicates (C2f), unresolved ambiguous rejections (C3), or stable or mismatched replays (C4).

In the planned cluster campaign, wasted work is reported as rejected attempts and their measured preparation CPU nanoseconds. Replayed responses are counted separately. The synthetic sink makes no provider calls; external-call waste is not inferred from database attempts. Failed and invalid trials are retained and reported. In that campaign, zero safety violations will be reported as violations divided by released at-risk batch attempts, with an exact one-sided 95% Clopper–Pearson upper bound. Continuous measures receive trial-level percentile-bootstrap 95% intervals with 10,000 resamples and seed 2026. Planned, valid and invalid n are recorded.

Planned takeover trials sweep $L \in \{5, 10, 15, 30\}$ s and use SIGKILL without lease release. A helper records a database-clock marker before killing the worker PID; the runner brackets kill time and verifies a higher restart count. The final renewal log yields a Δ estimate and interval; each row is compared to (4) using declared P, S, W budgets. Observed activation and sink times indicate whether those assumptions held. Graceful shutdown or immediate release would test a different path and is not mixed into this series.

Planned serving experiments calibrate μ at a declared p99 target, reject load-generator saturation, and compare $n_{\text{req}} - 1$ and n_{req} . Release experiments compare rolling, selector-switched blue-green and a client-weighted canary; this canary is not an Argo-controller benchmark. Green preparation and traffic-cutover intervals are recorded separately. Replica-seconds are an allocation proxy, not dollars. Table V fixes the endpoints.

TABLE V
EVALUATION ENDPOINTS

Metric	Definition	Raw source
Duplicates	Accepted repeated key	Sink attempts
Version regressions	Accepted source version < stored version; all conditions	Prior sink state
Epoch regressions	Accepted epoch < maximum applied epoch; lease conditions	Prior sink state
Late accepts	Old epoch after grant, before new sink write	Grant/accept logs
Wasted work	Rejections; preparation CPU	Sink attempts
Takeover	Kill bracket to first effect	Fault/accept logs
p99; violations	Latency; SLO failures	HTTP requests
Release cost	Extra replica-seconds	Readiness logs

C. Results

1) *Scripted sink-rule interleavings*: The exploratory study executes SQLite transactions under an imposed event schedule rather than the planned PostgreSQL/kind campaign. It encodes the co-located fence as a rule over caller-supplied holder and epoch values; it does not execute the PostgreSQL ownership functions. All items receive an initial v1 publication before the scripted grant to epoch 2. In each of 30 ordered schedules of 100 keys, the key order is shuffled and the successor publishes v3 first for each key with input probability 70%; otherwise the delayed former writer publishes v2 first. All conditions share each schedule’s seed. Thus a delayed v2 is a version regression only after a v3 publication. C1v rejects such attempts and accepts v2 when it precedes v3. C1, C1u and C2 allow version regressions when v3 arrives first. A feed-wide epoch metric additionally records former-epoch effects accepted after any successor write, even when the target item itself has not received v3.

Table VI reports the outcomes of these specified schedules. The co-located fence rejects the former holder before its submission reaches the state update. In C3r, the remote sink stores one highest epoch for the whole feed; any accepted epoch-2 write advances this fence for all item keys. Its late accepts occur only after the grant and before the successor’s first write reaches the remote sink. In the ordered schedule, successor and former writes are interleaved key by key in shuffled order, so the feed-wide remote window closes at the first successor write; C3r’s late count (9) is a property of that interleaving. The accepted delayed v2 writes are not version regressions when they precede v3. These differences are properties of the scripted interleavings and implemented rules, not observed production failure frequencies.

The pause duplicate series uses 1 deterministic scheduled run per applicable condition. The former epoch-1 writer repeats the original operation key after takeover: C1 and C2 accept this repeat, C1u rejects it by unique key, C2f/C3/C4 reject it by the co-located fence, and C3r rejects it by its unique key or feed fence. The separate lost-ack series

TABLE VI

SCRIPTED SQLITE INTERLEAVING OUTCOMES. COUNTS REFLECT THE SPECIFIED SCHEDULE AND SINK RULES, NOT FAILURE-RATE ESTIMATES.

Condition	n_D	n_O	Duplicates	Version reg.	Epoch reg.	Late	Rejected _O
C1	1	30	100	2,071	N/A	N/A	0
C1u	1	30	0	2,071	N/A	N/A	0
C1v	N/A	30	N/A	0	N/A	N/A	2,071
C2	1	30	100	2,071	2,991	N/A	0
C2f	1	30	0	0	0	N/A	3,000
C3	1	30	0	0	0	0	3,000
C3r	1	30	0	0	0	9	2,991
C4	1	30	0	0	0	N/A	3,000

n_D/n_O : scheduled duplicate/ordered runs; each run has 100 keys. Version and epoch regressions are separate outcomes on the ordered workload. Late means an accepted former-epoch write after grant and before the successor's first remote write. Rejected_O: delayed ordered attempts rejected by a guard. N/A: inapplicable or not measured. CPU and provider cost are not measured.

TABLE VII

SCRIPTED LOST-ACK RETRIES BY THE CURRENT OWNER. OUTCOME COUNTS FOLLOW THE CHOSEN LOSS DRAWS; ZERO RESPONSE MISMATCHES WERE OBSERVED.

Condition	Retry calls	Dup. accepted	Ambig. rejected	Stable replay
C2f	165	165	0	0
C3	165	0	165	0
C4	165	0	0	165

instead uses the *current* owner: across 30 paired seeded schedules per condition, 100 accepted calls each have input acknowledgment-loss probability 5%, and affected calls are retried with the same key after 50 ms. Table VII separates accepted duplicates, unresolved rejections and stable cached responses. No inferential interval is attached to an input loss probability chosen by the study.

For the after-grant event-time sweep, the successor's first remote write is placed at 2 s and the former writer resumes at $d \in \{0, 1, 5\}$ s. Each cell is one deterministic schedule of 100 keys. The former worker releases its complete captured batch at its scheduled resume time; the successor then publishes its own complete batch at the scheduled first-write time. When the former writer resumes before the successor's first remote write ($d = 0, 1$ s), the remote fence accepts its entire released batch (100 of 100 attempts); when it resumes afterward ($d = 5$ s), it rejects all of them. The co-located fence rejects every former-epoch attempt at every d . Figure 5 marks only the observed input delays and does not interpolate between them. These repeated keys share sink state, so the resulting zero counts do not justify independent-attempt binomial bounds. Throughput, takeover latency, preparation CPU, replica-seconds and SLO outcomes remain unmeasured.

D. Threats to validity and artifact availability

The planned kind cluster shares one host; CPU contention and host faults would correlate otherwise separate Pods. The completed SQLite study serializes an imposed schedule and cannot characterize concurrent execution, actual pause duration, clock uncertainty, external service behavior, or operational cost. Synthetic effects omit provider behavior, cache skew and session affinity. SIGSTOP captures a defined delayed

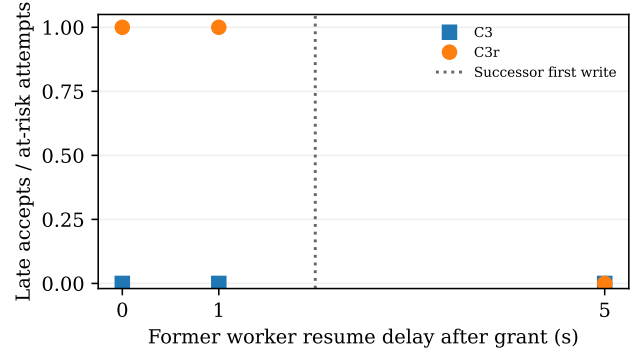


Fig. 5. Scripted after-grant late-accept fraction from raw trial rows. Markers represent the evaluated delays; the dotted line marks the scheduled successor first write. This is not a cluster timing measurement.

batch rather than every possible partition. PostgreSQL's transaction boundary is stronger than many external sinks. Cluster clock changes can affect expiry; bound interpretation requires clock and delay assumptions as well as a point estimate.

Artifact Availability: Source, manifests, SQL and protocol tests are publicly available at <https://github.com/nallabothukoteswar-wq/highlander-harness>. The experimental runner and analysis scripts remain under development. The exploratory raw CSVs are included in the repository; the planned cluster campaign has not generated data and an archival DOI remains unavailable.

VI. DISCUSSION AND LIMITATIONS

Idempotency keys solve repeatable, keyable effects. A conditional version check blocks version regressions when source versions are comparable. Epoch fencing blocks epoch authority regressions for lease conditions when a lower epoch follows a higher one; a remote epoch fence can still admit an older epoch after a successor grant but before its first remote write. These metrics must be analyzed separately. Ownership can reduce contention and wasted work but does not prove that only one process is executing. An ordered workload must also define order within one epoch; our test worker publishes sequentially. If current-epoch operations can be reordered in transit, an additional version check is required. A fencing

result must not be generalized to ordering that the sink never checks.

A payment or email provider remains outside the local transaction: an outbox preserves intent but cannot settle an unknown external outcome alone. Per-key ownership avoids a global bottleneck for independent feeds. Interactive APIs retain active-active capacity sizing even when their effect paths are serialized. A low-traffic internal interface is single-active only when capacity and availability constraints both pass. During coordinator loss, stopping effects favors safety at the cost of progress. Blue-green rollback needs compatible state; canaries need representative traffic and version-tagged metrics. PodDisruptionBudgets restrict voluntary disruptions, not node loss [25].

The analytical results establish a decision decomposition and a conditional sink safety argument. The scripted SQLite outcomes expose distinct version, epoch, and remote late-accept cases under their chosen schedules; they do not establish production speedup, cost savings, or failure-rate reduction. The supplied PostgreSQL protocol tests target the unique-only, fence-only, atomic-acquisition, response-replay and restricted-remote-sink cases but were not executed here.

VII. CONCLUSION AND FUTURE WORK

Workload-aware deployment requires separate decisions for serving topology, effect discipline and release mechanism. Atomic ownership binds a holder to its fencing epoch, while atomic operation-key handling prevents repeat effects. The distinction is necessary to evaluate Highlander as a single-effective-writer policy without attributing uniqueness guarantees to leadership. Future work should assess production traces, multi-zone coordinator faults, reordered current-epoch publications and real provider idempotency contracts. The reported sink-rule observations are limited to scripted, serialized SQLite interleavings; concurrent and production claims require the planned cluster campaign.

REFERENCES

- [1] Kubernetes Authors, “Deployments,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/controllers/deployment/> (accessed Sep. 26, 2026).
- [2] M. Fowler, “Blue Green Deployment,” 2010. [Online]. Available: <https://martinfowler.com/bliki/BlueGreenDeployment.html> (accessed Sep. 26, 2026).
- [3] M. Fowler, “Canary Release,” 2014. [Online]. Available: <https://martinfowler.com/bliki/CanaryRelease.html> (accessed Sep. 26, 2026).
- [4] C. G. Gray and D. R. Cheriton, “Leases: An Efficient Fault-Tolerant Mechanism for Distributed File Cache Consistency,” in *Proc. ACM SOSP*, 1989. <https://doi.org/10.1145/74850.74870>
- [5] M. Burrows, “The Chubby Lock Service for Loosely-Coupled Distributed Systems,” in *Proc. USENIX OSDI*, 2006. <https://www.usenix.org/conference/osdi-06/chubby-lock-service-loosely-coupled-distributed-systems>
- [6] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, “ZooKeeper: Wait-free Coordination for Internet-scale Systems,” in *Proc. USENIX ATC*, 2010. <https://www.usenix.org/conference/usenix-atc-10/zookeeper-wait-free-coordination-internet-scale-systems>
- [7] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” in *Proc. USENIX ATC*, 2014. <https://www.usenix.org/conference/atc14/technical-sessions/presentation/ongaro>
- [8] M. Kleppmann, *Designing Data-Intensive Applications*. O’Reilly Media, 2017. <https://dataintensive.net/>
- [9] P. Helland, “Idempotence Is Not a Medical Condition,” *ACM Queue*, 2012. <https://queue.acm.org/detail.cfm?id=2187821>
- [10] H. Garcia-Molina and K. Salem, “Sagas,” in *Proc. ACM SIGMOD*, 1987. <https://doi.org/10.1145/38713.38742>
- [11] A. Verma *et al.*, “Large-scale cluster management at Google with Borg,” in *Proc. ACM EuroSys*, 2015. <https://research.google/pubs/large-scale-cluster-management-at-google-with-borg/>
- [12] M. Schwarzkopf, A. Konwinski, M. Abd-El-Malek, and J. Wilkes, “Omega: flexible, scalable schedulers for large compute clusters,” in *Proc. ACM EuroSys*, 2013. <https://research.google/pubs/omega-flexible-scalable-schedulers-for-large-compute-clusters/>
- [13] M. Tirmazi *et al.*, “Borg: the Next Generation,” in *Proc. ACM EuroSys*, 2020. <https://doi.org/10.1145/3342195.3387517>
- [14] T. Akidau *et al.*, “MillWheel: Fault-Tolerant Stream Processing at Internet Scale,” *Proc. VLDB Endowment*, 2013. <https://research.google/pubs/millwheel-fault-tolerant-stream-processing-at-internet-scale/>
- [15] R. Ananthanarayanan *et al.*, “Photon: Fault-tolerant and Scalable Joining of Continuous Data Streams,” in *Proc. ACM SIGMOD*, 2013. <https://research.google/pubs/photon-fault-tolerant-and-scalable-joining-of-continuous-data-streams/>
- [16] G. Schermann, J. Cito, P. Leitner, U. Zdun, and C. H. Gall, “We’re doing it live: A multi-method empirical study on continuous experimentation,” *Information and Software Technology*, vol. 99, 2018. <https://eprints.cs.univie.ac.at/5953/>
- [17] Š. Davidović and B. Beyer, “Canary Analysis Service,” *ACM Queue*, vol. 16, no. 1, 2018. <https://queue.acm.org/detail.cfm?id=3194655>
- [18] Kubernetes Authors, “Jobs,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/controllers/job/> (accessed Sep. 26, 2026).
- [19] Kubernetes Authors, “Liveness, Readiness, and Startup Probes,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/pods/probes/> (accessed Sep. 26, 2026).
- [20] Kubernetes Authors, “Leases,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/concepts/architecture/leases/> (accessed Sep. 26, 2026).
- [21] Kubernetes Authors, “Horizontal Pod Autoscaling,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/> (accessed Sep. 26, 2026).
- [22] Argo Project, “Canary Deployment Strategy,” *Argo Rollouts Documentation*. [Online]. Available: <https://argo-rollouts.readthedocs.io/en/stable/features/canary/> (accessed Sep. 26, 2026).
- [23] Kubernetes Authors, “leaderelection package,” client-go source documentation. [Online]. Available: <https://pkg.go.dev/k8s.io/client-go/tools/leaderelection> (accessed Sep. 26, 2026).
- [24] Linux man-pages project, “pid_namespaces(7),” *Linux manual page*. [Online]. Available: https://man7.org/linux/man-pages/man7/pid_namespaces.7.html (accessed Sep. 26, 2026).
- [25] Kubernetes Authors, “Specifying a Disruption Budget for your Application,” *Kubernetes Documentation*. [Online]. Available: <https://kubernetes.io/docs/tasks/run-application/configure-pdb/> (accessed Sep. 26, 2026).